

LAPORAN TUGAS 1

VM MANAGER

CSCM602055 - Sistem Operasi B

Kelompok B3 - Varian B



Fakultas Ilmu Komputer

Universitas Indonesia

Gasal 2026/2027

1. Identitas Kelompok

Tugas ini dikerjakan oleh Kelompok B3 pada kelas Sistem Operasi B. Pembagian tanggung jawab utama tiap anggota disusun berdasarkan bagian implementasi dan demonstrasi yang dikerjakan.

No.	Nama	NPM	Tanggung Jawab Utama
1	Faris Salman Azhari	2506615223	Integrasi sistem, sysinfo_report.txt, fitur uptime, dan penyusunan laporan tugas.
2	Syabil Wafi Ahdi	2506657371	Host lifecycle pada vm_ctl.sh: list, info, start, dan stop.
3	Dyah Zhafira Wibowo	2506623723	Fitur snapshot pada host dan basic system information pada guest.
4	Silvia Lalita Damayanti	2506621863	Metrik Varian B, resource_check.c, serta komunikasi pipe/stdin.

2. Gambaran Umum Program

VM Manager merupakan tool sederhana untuk mengendalikan dan memantau Virtual Machine Ubuntu yang berjalan melalui VirtualBox. Sistem dibagi menjadi dua sisi. Sisi host menggunakan vm_ctl.sh dan VBoxManage untuk mengendalikan siklus hidup VM serta snapshot, sedangkan sisi guest menggunakan sysinfo.sh dan resource_check.c untuk mengambil informasi sistem dan mengevaluasi kondisi resource.

Alur utama sistem dimulai dari pengambilan data oleh Bash. Untuk Varian B, sysinfo.sh menghitung memory usage dan load average terhadap jumlah CPU core. Kedua nilai tersebut dikirim melalui pipe ke standard input program resource_check.c. Program C menentukan status PASS, WARN, atau FAIL sesuai threshold yang ditetapkan, lalu output-nya ditangkap kembali oleh sysinfo.sh dan dimasukkan ke dalam sysinfo_report.txt.

Berkas	Dijalankan di	Fungsi
vm_ctl.sh	Host	Mengendalikan VM melalui VBoxManage: list, info, start, stop, snapshot create, dan snapshot list.

sysinfo.sh	Guest	Mengambil informasi sistem, menghitung metrik Varian B, menjalankan resource_check, dan membentuk laporan.
resource_check.c	Guest	Membaca dua metrik melalui stdin dan menentukan status PASS/WARN/FAIL.
sysinfo_report.txt	Guest	Menyimpan hasil akhir pemeriksaan sistem dalam format tabel teks.

3. Implementasi Sisi Host - vm_ctl.sh

Script vm_ctl.sh dijalankan dari komputer host melalui terminal Bash. Pada host Windows, script menambahkan lokasi instalasi VirtualBox ke PATH agar perintah VBoxManage dapat dipanggil. Struktur script dibuat modular dengan function terpisah untuk setiap operasi utama.

3.1 List VM

Function list_vms() menjalankan VBoxManage list vms untuk mengambil seluruh Virtual Machine yang terdaftar. Nama VM kemudian diproses dan ditampilkan dalam daftar bernomor agar mudah dibaca.

```
./vm_ctl.sh list
```

3.2 Informasi VM

Function info_vm() menggunakan VBoxManage showvminfo dengan output machine-readable. Script mengambil nilai memory, cpus, dan VMState untuk menampilkan RAM yang dialokasikan, jumlah vCPU, serta status VM saat ini. Script juga memeriksa bahwa nama VM telah diberikan dan VM tersebut dapat ditemukan.

```
./vm_ctl.sh info VM-Manager
```

3.3 Menyalakan VM

Function start_vm() menjalankan VM dengan mode headless menggunakan VBoxManage startvm. Mode headless memungkinkan VM berjalan di background tanpa membuka jendela antarmuka VirtualBox.

```
./vm_ctl.sh start VM-Manager
```

3.4 Mematikan VM dengan Aman

Function stop_vm() tidak menggunakan force power off. Script mengirim sinyal ACPI power button menggunakan VBoxManage controlvm sehingga Ubuntu dapat melakukan shutdown secara normal. Setelah sinyal dikirim, script memeriksa status VM secara berkala hingga status berubah menjadi poweroff atau batas waktu pengecekan tercapai.

```
./vm_ctl.sh stop VM-Manager
```

3.5 Snapshot

Function `snapshot_create()` membuat snapshot baru berdasarkan nama VM dan nama snapshot yang diberikan. Function `snapshot_list()` mengambil daftar snapshot yang tersedia dan menampilkan nama masing-masing snapshot secara berurutan.

```
./vm_ctl.sh snapshot create VM-Manager <nama_snapshot>
```

```
./vm_ctl.sh snapshot list VM-Manager
```

4. Implementasi Sisi Guest - sysinfo.sh

Script `sysinfo.sh` dijalankan di dalam Ubuntu VM. Script ini menggabungkan pemeriksaan informasi sistem dasar, pengambilan metrik Varian B, evaluasi resource melalui program C, fitur tambahan uptime, dan pembentukan laporan akhir.

4.1 Informasi Sistem Dasar

Empat informasi dasar yang diperiksa adalah OS dan kernel, jumlah akun pengguna biasa, jumlah proses berjalan, dan status virtualisasi. Nama OS dibaca dari `/etc/os-release` dan kernel dari `uname -r`. Regular user dihitung dari `/etc/passwd` dengan rentang UID 1000 sampai kurang dari 60000. Jumlah proses diperoleh dari `ps -e`, sedangkan deteksi virtualisasi menggunakan `systemd-detect-virt`.

4.2 Metrik 1 - Memory Usage

Untuk Varian B, metrik pertama adalah persentase penggunaan memory. Script menggunakan perintah `free -m` untuk mengambil total memory dan available memory. Persentase penggunaan kemudian dihitung dengan rumus berikut.

$$\text{Memory Usage} = ((\text{Total Memory} - \text{Available Memory}) / \text{Total Memory}) \times 100\%$$

Threshold yang digunakan adalah PASS untuk nilai di bawah 75%, WARN untuk nilai mulai 75% sampai kurang dari 90%, dan FAIL untuk penggunaan memory yang mencapai atau melebihi 90%.

4.3 Metrik 2 - Load Average terhadap CPU Core

Metrik kedua menggunakan load average satu menit dari `/proc/loadavg` dan jumlah CPU core dari `nproc`. Nilai load average kemudian dibagi dengan jumlah core untuk memperoleh load per core.

$$\text{Load/Core Ratio} = \text{Load Average 1 Minute} / \text{Jumlah CPU Core}$$

Status PASS diberikan ketika load/core tidak lebih dari 1. Nilai di atas 1 sampai 2 diberi status WARN, sedangkan nilai di atas 2 diberi status FAIL. Alasan penggunaan batas ini adalah bahwa load per core sekitar 1 menunjukkan jumlah pekerjaan aktif sudah sebanding dengan kapasitas core yang tersedia. Ketika nilainya melewati 1, mulai terdapat tekanan atau antrean kerja, sedangkan nilai di atas 2 menunjukkan beban yang jauh lebih besar dibanding kapasitas relatif CPU.

4.4 Komunikasi Pipe ke Program C

Kedua nilai metrik tidak dikirim sebagai command-line argument. sysinfo.sh menuliskan memory_usage dan load_ratio ke standard output, kemudian meneruskannya melalui pipe ke standard input program resource_check. Output program C disimpan pada variabel result dan status akhir tiap metrik diambil dari output tersebut menggunakan awk.

```
result=$(echo "$memory_usage $load_ratio" | ./resource_check)
```

5. Implementasi resource_check.c

Program resource_check.c membaca dua angka bertipe double dari standard input menggunakan scanf. Apabila input tidak valid, program mencetak ERROR dan berhenti dengan return code 1. Jika input valid, program menggunakan percabangan if-else untuk menentukan status masing-masing metrik sesuai threshold Varian B.

Metrik	PASS	WARN	FAIL
Memory Usage	< 75%	>= 75% dan < 90%	>= 90%
Load/Core	<= 1	> 1 dan <= 2	> 2

Program mencetak hasil dalam dua baris, yaitu status untuk Metric 1 (Memory) dan Metric 2 (Load/Core). sysinfo.sh kemudian membaca field status tersebut untuk digunakan pada tampilan terminal dan laporan akhir.

6. Fitur Tambahan - VM Uptime

Sebagai fitur tambahan di luar requirement utama, Kelompok B3 menambahkan informasi VM uptime pada sisi guest. Informasi ini diperoleh menggunakan perintah uptime -p dan menunjukkan berapa lama Ubuntu VM telah berjalan sejak proses boot terakhir.

Fitur uptime dipilih karena relevan dengan fungsi monitoring. Data ini dapat membantu pengguna mengetahui durasi VM telah aktif dan memberikan konteks tambahan ketika membaca kondisi resource pada saat pemeriksaan dilakukan.

7. Pembentukan sysinfo_report.txt

Setelah seluruh pemeriksaan selesai, sysinfo.sh membentuk file sysinfo_report.txt dalam format tabel teks. Laporan memuat kategori pemeriksaan, nilai atau item yang diperoleh, status, dan detail. Untuk Memory dan Load/Core, status yang ditulis ke report berasal dari output resource_check.c sehingga tidak dihitung ulang secara terpisah oleh Bash.

Check Category	Item	Status	Details
----------------	------	--------	---------

OS	Ubuntu 24.04 LTS (Kernel 7.0.0-30-generic)	PASS	OS dan Kernel
Users	1 akun	PASS	Regular accounts
Processes	113 proses	PASS	Running processes
Virtualization	Terdeteksi (VirtualBox)	PASS	Hypervisor
Memory	13.83%	PASS	Memory usage
Load/Core	0.00	PASS	Load 0.00 / 2 cores
Uptime	52 minutes	PASS	VM uptime

8. Alur Pengujian dan Eksekusi

Urutan berikut digunakan untuk memastikan seluruh fungsi utama dapat diuji secara berurutan tanpa mematikan VM sebelum pemeriksaan guest selesai.

8.1 Sisi Host

```
./vm_ctl.sh list
./vm_ctl.sh info VM-Manager
./vm_ctl.sh start VM-Manager
./vm_ctl.sh snapshot create VM-Manager demo-snapshot
./vm_ctl.sh snapshot list VM-Manager
```

8.2 Sisi Guest

```
chmod +x sysinfo.sh
gcc resource_check.c -o resource_check
printf "80 1.5\n" | ./resource_check
./sysinfo.sh
cat sysinfo_report.txt
```

8.3 Kembali ke Host

```
./vm_ctl.sh stop VM-Manager
```

9. Video Presentasi

Link video: [LINK VIDEO UNLISTED YOUTUBE](#)

Link video: [LINK VIDEO UNLISTED YOUTUBE](#)

Link video: [LINK VIDEO UNLISTED YOUTUBE](#)

10. Kesimpulan

VM Manager Kelompok B3 telah mengintegrasikan kontrol Virtual Machine dari sisi host dengan monitoring resource dari sisi guest. `vm_ctl.sh` menangani lifecycle dan snapshot melalui `VBoxManage`, sedangkan `sysinfo.sh` mengambil informasi sistem serta metrik Varian B. Evaluasi Memory dan Load/Core dilakukan oleh `resource_check.c` melalui mekanisme pipe dan standard input, kemudian hasilnya digabungkan bersama fitur uptime ke dalam `sysinfo_report.txt`.

Dengan pembagian ini, seluruh komponen dapat diuji sebagai satu alur end-to-end: VM dinyalakan dari host, diperiksa dari guest, hasil resource dievaluasi oleh program C, laporan akhir dibuat, dan VM kemudian dimatikan kembali dengan mekanisme shutdown yang aman.